ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Encapsulation Challenge — Feedback

This document collects the points that emerged from the in-class discussion of the encapsulation challenge, together with reference solutions and the design choices that were available at each step. Use it to revisit your group's work, compare it with the alternatives, and reflect on the design decisions you made (or skipped).

Designing the interface (Part A)

Desiging the interface involves a number of decisions. Some of them are genuinely open — different groups will have made different choices, and any of several answers is defensible. Others are non-negotiable: getting them wrong undermines the whole exercise.

Non-negotiable choices

Types Exported: Both `Wallet` and `Money` : they are the abstractions provided to clients.

Are the data constructors exported? No. This is the whole point of the exercise. Writing `module Wallet (Wallet(..), Money(..), ...)` , the `(..)` would export the constructors as well as the type names — and the abstraction is gone. Constructors must stay private to the module.

Does `spend` return a new wallet? Yes. Spend should change the state of the wallet, to keep track that the `Money` has been spent, so as to enforce the no-double-spending invariant is unenforceable. Given that Haskell is purely functional, returning a new wallet is the simplest way to model the state change (alternatively, we would need to resort to *monads*, but they are out our our present scope).

Does `mint` return a new wallet? Yes, for the same reason: without returning the updated wallet, fresh ids cannot be tracked across calls.

Open choices

How should the result of `spend` be returned? `spend` either succeeds or fails (because the token has already been spent): then it succeeds, in addition to the updated wallet, `spend` may return the new wallet balance. To capture the two possible outcomes, the natural choice is to use a variant. Here's three possible alternative:

- `Either String (Wallet, Int)` — successful spends return the updated wallet and the updated wallet's balance; failures carry an error message string.
- `Maybe (Wallet, Int)` — same shape on success, but failure is just `Nothing` — no reason given.
- A custom sum type like data `SpendResult = Spent Wallet Int | AlreadySpent` — most informative, most verbose, makes the failure case named and pattern-matchable.

Any of the three is a reasonable choice. The point is to *notice* the trade-off: error messages are helpful for debugging, but they do require some commitment to a string type.

Is `verify` part of the interface? Strictly speaking it is not necessary — you can attempt to spend and recover from the failure. Including it gives the client a non-destructive way to ask "is this `Money` still good?" without consuming it. It also illustrates a subtle point: an abstraction can expose *queries* on hidden state without exposing the state itself.

Is `balance` part of the interface? Exposing the total amount of unspent money is harmless from a security standpoint — it does not enable double-spending — but it does leak some accounting information. Leaving it out makles learning the amount possible only when spending. That is also defensible.

Reference interface

```
module Wallet (Wallet, Money, empty, mint, spend, verify, balance) where

empty   :: Wallet                                  -- a fresh, empty wallet
mint    :: Wallet -> Int -> (Wallet, Money)        -- create new Money of a given amount
spend   :: Wallet -> Money -> Either String (Wallet, Int)
                                                   -- spend Money: returns new balance, or an er
verify  :: Wallet -> Money -> Bool                 -- is this Money still spendable?
balance :: Wallet -> Int                           -- total amount of unspent Money
```

Reference solution — Haskell

For the `Wallet` implementation we'll choose a sligthly different starting point than the one suggested in the challenge: instead of carrying the set of spent ids, we'll keep two separate lists, of spent and unspent money.

```haskell
module Wallet (Wallet, Money, empty, mint, spend, verify, balance) where

-- Internal representations. Constructors NOT exported.

data Money  = MkMoney  { moneyId :: Int, moneyAmount :: Int }
data Wallet = MkWallet { unspent :: [Money]
                       , spent   :: [Money]
                       , nextId  :: Int     -- counter for generating fresh ids
                       }

empty :: Wallet
empty = MkWallet [] [] 0

mint :: Wallet -> Int -> (Wallet, Money)
mint (MkWallet u s next) amount =
  let m  = MkMoney next amount
      w' = MkWallet (m : u) s (next + 1)
  in (w', m)

spend :: Wallet -> Money -> Either String (Wallet, Int)
spend (MkWallet u s next) m =
  let i = moneyId m
  in case break (\m' -> moneyId m' == i) u of
       (before, found : after) ->
         Right (MkWallet (before ++ after) (found : s) next, moneyAmount found)
       _ ->
         if any (\m' -> moneyId m' == i) s
           then Left "already spent"
           else Left "unknown token"

verify :: Wallet -> Money -> Bool
verify (MkWallet u _ _) m = any (\m' -> moneyId m' == moneyId m) u

balance :: Wallet -> Int
balance MkWallet{unspent = u} = sum . map moneyAmount $ u
```

The implementaion uses a few idiomatic Haskell forms:

- `break :: (a -> Bool) -> [a] -> ([a], [a])` : a standard Haskell function which splits a list into two parts at the first element that satisfies the predicate. The first part contains all elements before the match (none of which satisfy the predicate); the second part starts with the matching element and contains everything from there to the end.
- `any :: (a -> Bool) -> [a] -> Bool` : returns returns `True` if at least one element of the list statisfies the predicate;
- record field selectors, using record labels ad functions (cf. lecture notes on Algebraic Data Types);
- different ways to express constructor patterns: positional `MkWallet u s next` vs record `MkWallet{unspent = u}` . Both are legal positional is less verbose, but record is more rebust, as positional may end up silently match wrong fields.

A small client to exercise it:

```
module Main where

import Wallet

example :: String
example =
  let w0       = empty
      (w1, m1) = mint w0 100
      (w2, m2) = mint w1  50
  in case spend w2 m1 of
       Left e -> "ERROR: first spend failed: " ++ e
       Right (w3, amt1) ->
         "first spend ok: " ++ show amt1 ++ "\n" ++
         case spend w3 m1 of
           Right _ -> "ERROR: double spend succeeded!"
           Left  e -> "second spend correctly rejected: " ++ e

main :: IO ()
main = putStrLn example
```

Expected output:

```
first spend ok: 100
second spend correctly rejected: already spent
```

What the attacker cannot do (Haskell)

In a *client* file `Client.hs` that does `import Wallet`:

```
-- (a) Fabricate a Money value.
fake :: Money
fake = MkMoney 999 1000000
-- error: Data constructor not in scope: MkMoney


-- (b) Pattern-match on a Money to read its id.
peek :: Money -> Int
peek (MkMoney i _) = i
-- error: Not in scope: data constructor 'MkMoney'


-- (c) Fabricate a Wallet with a manipulated spent set.
forge :: Wallet
forge = MkWallet [] [] 0
-- error: Data constructor not in scope: MkWallet
```

All three fail at compile time because the export list of the `Wallet` module exports only the *type names* `Wallet` and `Money`, not their data constructors `MkWallet` and `MkMoney`. From the client's point of view the types are fully opaque: there is no syntactic way to construct or destructure a value of these types except through the exported operations. The internal `Set Int` is unreachable.

Reference Implementation in Scala

We provide two solutions. The first, below, is the direct translation of the reference Haskell interface and implementation.

```
object Wallets:

  // Internal representations. Outside this object they are opaque.
  opaque type Money  = (Int, Int)                              // (id, amount)
  opaque type Wallet = (List[Money], List[Money], Int)         // (unspent, spent, next id)

  // Where empty lives — a companion-object-style entry point.
  object Wallet:
    def empty: Wallet = (List.empty, List.empty, 0)

  // Operations expressed as extension methods.
  extension (w: Wallet)
    def mint(amount: Int): (Wallet, Money) =
      val (unspent, spent, next) = w
      val m: Money               = (next, amount)
      val w2: Wallet             = (m :: unspent, spent, next + 1)
      (w2, m)

    def spend(m: Money): Either[String, (Wallet, Int)] =
      val (unspent, spent, next) = w
      val (id, amount)           = m
      unspent.find(_._1 == id) match
        case Some(found) =>
          val unspent2 = unspent.filterNot(_._1 == id)
          Right(((unspent2, found :: spent, next), unspent2.map(_._2).sum))
        case None =>
          if spent.exists(_._1 == id) then Left("already spent")
          else Left("unknown token")

    def verify(m: Money): Boolean =
      val (unspent, _, _) = w
      unspent.exists(_._1 == m._1)

    def balance: Int =
      val (unspent, _, _) = w
      unspent.map(_._2).sum
```

A client:

```
@main def example(): Unit =
  import Wallets.*

  val w0       = Wallet.empty
  val (w1, m1) = w0.mint(100)
  val (w2, m2) = w1.mint(50)

  w2.spend(m1) match
    case Right((w3, amt1)) =>
      println(s"first spend ok: $amt1")
      w3.spend(m1) match
        case Right(_) => println("ERROR: double spend succeeded!")
        case Left(e)  => println(s"second spend correctly rejected: $e")
    case Left(e) => println(s"ERROR: first spend failed: $e")
```

The functional-style alternative (standalone functions) reads the same way modulo `mint(w, 100)` instead of `w.mint(100)`. Either is fine; the choice is stylistic.

Idiomatic choices made here

- **Extension methods over standalone functions.** The chained syntax `w0.mint(100)` reads more naturally than `mint(w0, 100)` when many operations are threaded — and in a state-passing style, that is most calls. A pure functional style would be equally defensible.
- **`Either[String, ...]` for failure.** Symmetric with the Haskell version. `Option` would lose the error message; a sealed trait would be more verbose without buying anything for a single failure mode.
- **`empty` inside `object Wallet`.** Lets clients write `Wallet.empty`, parallel to the way Haskell users would import the module-qualified `Wallet.empty`. A top-level `def emptyWallet` would also work but reads less idiomatically.

What the attacker cannot do (Scala)

From outside `Wallets`:

```
val m: Wallets.Money = (999, 1000000)
// error: Found: (Int, Int), Required: Wallets.Money

val (id, amt) = m
// error: cannot match against opaque type

val w: Wallets.Wallet = (Nil, Nil, 0)
// error: Found: (Nil.type, Nil.type, Int), Required: Wallets.Wallet
```

Outside the defining `object`, the opaque types are abstract: the type checker refuses any operation that depends on the underlying representation. Inside the `object`, the equivalence is restored — the implementation can freely pattern-match on the tuples.

Comparing the two enforcement mechanisms

Both Haskell's export list and Scala's `opaque type` produce the same effect at the type level: outside the defining module, the representation is unobservable. Two differences are worth keeping in mind:

- **Granularity.** Haskell's export list is module-wide and applies to data constructors uniformly. Scala's `opaque` is per-type and lives inside the defining `object`, which makes it easy to mix opaque and transparent definitions in the same file.
- **Zero-cost abstraction.** Scala's opaque types compile to the underlying representation with no wrapper; there is no runtime overhead. Haskell `newtype` has the same property, but a `data` declaration introduces a wrapper at runtime. In this challenge we used `data` for clarity, but a real-world version would use `newtype` for `Money`.

In both languages the abstraction is enforced statically: any violation is a compile error, not a runtime check. Compare with Elixir, where `@opaque` is enforced only by the optional Dialyzer analysis.

Scala Stateful Implementation

A much more idiomatic Scala implementation would mutate the wallet internal lists in place, and clients wouldn't need to thread it through every operation.

```
object Wallets:

  opaque type Money = (Int, Int)   // (id, amount)

  class Wallet:
    private var unspent: List[Money] = Nil
    private var spent:   List[Money] = Nil
    private var nextId:  Int         = 0

    def mint(amount: Int): Money =
      val m: Money = (nextId, amount)
      unspent = m :: unspent
      nextId += 1
      m

    def spend(m: Money): Either[String, Int] =
      val (id, amount) = m
      unspent.find(_._1 == id) match
        case Some(found) =>
          unspent = unspent.filterNot(_._1 == id)
          spent   = found :: spent
          Right(amount)
        case None =>
          if spent.exists(_._1 == id) then Left("already spent")
          else Left("unknown token")

    def verify(m: Money): Boolean =
      unspent.exists(_._1 == m._1)

    def balance: Int =
      unspent.map(_._2).sum

  object Wallet:
    def empty: Wallet = new Wallet
```

Now the client code reads naturally:

```
scalaval w  = Wallet.empty
val m1 = w.mint(100)
val m2 = w.mint(50)
w.spend(m1)            // Right(100)
w.spend(m1)            // Left("already spent")
w.balance              // 50
```

No more threading. Each operation just mutates the wallet and returns the result. Encapsulation is now provided by:

- `Wallet` a class (not an opaque type), so new Wallet is the only way to create one — but it's hidden behind Wallet.empty
- The var fields are private, so clients cannot inspect or modify them
- `Money` is still opaque

The trade-off: we have given up purity (the wallet is now stateful) for ease of interaction (no threading). This is exactly the OO style that Scala makes natural and Haskell deliberately avoids.

The take-away is: Haskell's purity has costs (the threading awkwardness), and Scala's flexibility lets you escape those costs at the price of a different reasoning model. Neither is unambiguously better.

Reflection (Part C)

Why is `Wallet` the more security-critical abstraction?

`Money` is hidden mainly to prevent *forgery*: if `MkMoney` were exported, a client could construct a `Money` with any id and amount of their choosing and present it for spending. But the no-double-spending invariant lives in the `Wallet` 's spent set. If the `Wallet` representation were public, a client could construct a fresh `Wallet` with the spent set emptied and use it to "un-spend" any `Money` they had already presented.

Concretely:

- `Money` private + `Wallet` public → no forgery, but trivial double-spending.
- `Money` public + `Wallet` private → forgery is possible (just construct a `Money` with whatever id), but at least the wallet remembers what has been spent.
- Both private → the invariant holds.

A useful framing: `Money` is a *capability* (a token you possess), `Wallet` is the *authority* that decides whether a capability can still be exercised. Both need encapsulation, but the security property depends

primarily on the authority being abstract.

Why is wallet threading awkward?

Every operation that changes state takes a wallet and returns a new one, and the client must thread it through manually. This is exactly the pain that *state monads* address: a state monad is a discipline for hiding the threading inside a binding operator, so the client writes `mint 100; spend m1` and the wallet flows through implicitly. We will return to this when we discuss monads.

ADTs alone vs ADTs plus encapsulation

ADTs alone get you the *structure* — distinguished cases, payload types, exhaustive pattern matching. What they do not get you is *protection*: if `Wallet` and `Money` are public ADTs, nothing prevents a client from constructing a `Wallet` with whatever spent set they like, or pattern-matching to forge a `Money`. ADTs structure the data; encapsulation protects the data. They are complementary, not alternatives.

A precise way to put it: the no-double-spending invariant is a property of the *operations*, not of the data. ADTs let you describe data; only encapsulation lets you constrain what operations clients can perform on it.

An Alternative design -- Haskell

A different way to approach the implementation is to encode `Money` as a pair where the first component is a boolean that informs on whether the amount encoded by the second component has been spent or not.

Here's the rewritten module:

```haskell
module Wallet (Wallet, Money, empty, mint, spend, verify, balance) where

data Money  = MkMoney  { spent :: Bool, amount :: Int }
data Wallet = MkWallet { unspent :: Int, totalSpent :: Int }

empty :: Wallet
empty = MkWallet 0 0

mint :: Wallet -> Int -> (Wallet, Money)
mint (MkWallet u s) n =
  let m  = MkMoney False n
      w' = MkWallet (u + n) s
  in (w', m)

spend :: Wallet -> Money -> Either String (Wallet, Int)
spend (MkWallet u s) m
  | spent m       = Left "already spent"
  | u < amount m  = Left "insufficient funds"
  | otherwise     =
      let w' = MkWallet (u - amount m) (s + amount m)
      in Right (w', u - amount m)

verify :: Wallet -> Money -> Bool
verify _ m = not (spent m)

balance :: Wallet -> Int
balance (MkWallet u _) = u
```

Notes on the alternative design:

- **`Wallet` carries totals, not individual tokens.** When you `mint` a `Money 50`, the wallet's `unspent` goes up by 50. When you `spend` it, `unspent` goes down by 50 and `totalSpent` goes up by 50.
- **The `spent` flag on `Money` prevents the obvious re-presentation.** If a client presents the same `Money` value twice, the second spend fails because the flag is set.
- **`balance` is just `unspent`.** No iteration, no filtering — it's already the total.
- **`spend` checks `unspent < amount m`.** This is a real check now: if the wallet has been drained, you can't spend any more, even with an unflagged `Money`.

This is much cleaner than the id-based version, and the no-double-spending property degrades gracefully into "no over-spending" — the wallet is the source of truth about how much value is still available. The flag on `Money` is a polite cooperative signal; the wallet's totals are the enforcer.